



**Politecnico
di Torino**

Cybersecurity for Embedded Systems

Project #2: Task Isolation in FreeRTOS
Documentation and Implementation

GROUP #9

Gabriele Noè - s364766

Luca Pittalis - s358755

Sergio Princivalle - s364116

Lorenzo Russo - s358556

June 26, 2026

Contents

1	Introduction	1
1.1	The Importance of Task Isolation in RTOS	1
1.2	Demonstration Purposes and Testing Repository	2
1.2.1	General Testing Environment Information	2
1.2.2	Folder Tree Structure	2
1.2.3	Demonstration Objectives	2
1.3	Ideal Scenario, Assumptions and Broken Rules	3
1.3.1	Assumptions	3
1.3.2	Broken Rules	3
2	Attack without the MPU	4
2.1	Step-by-Step Execution Scenario	4
2.1.1	Target task (victim)	4
2.1.2	Global Data Attacker task	4
2.1.3	Stack Attacker task	4
2.2	Result of Task Isolation Failure	7
3	Implementation of the MPU	8
3.1	What is an MPU?	8
3.2	Unit Implementation	10
3.2.1	First implementation and issues with global variables	10
3.2.2	Second implementation and issues with <code>snprintf</code>	11
3.2.3	Final working implementation	12
3.3	Overview of Stack after Implementation	13
3.3.1	Linker Script Isolation and the Power-of-Two Alignment Rule	13
3.3.2	MPU Enforcement and Attack Vector Demonstration	13
3.3.3	<code>_snprintf_r</code> and the Stack Footprint	13
4	Attack with the MPU	14
4.1	Attack Implementation	14
4.1.1	First Attack: Stack Scanning and TCB Introspection Failure	14
4.1.2	Second Attack: Shared Global Variable Access Protection	14
4.1.3	Transition from Dynamic to Static Task Buffer Allocation and MPU Setup	14
4.2	Step-by-Step Execution Scenario	15
4.3	Result of Task Isolation Failure	15
4.3.1	<code>priorityAndStackAttacker</code> Exception Log	15
4.3.2	<code>globalVarAttacker</code> Exception Log	16
5	Conclusions	17
5.1	Summary of the Experience	17
5.2	Sources and References	17

CHAPTER 1

Introduction

1.1 The Importance of Task Isolation in RTOS

Modern Real-Time Operating Systems are commonly used in embedded applications where multiple functionalities must execute concurrently while respecting strict timing constraints. To achieve this, the application is typically divided into independent tasks, each responsible for a specific operation and managed by the RTOS scheduler.

Although tasks execute within the same system, they often process data with different levels of sensitivity and perform operations with different security requirements. For example, one task may handle cryptographic material, another may manage communications, while a third may collect sensor data or update shared system state. In such environments, a fundamental security requirement is **task isolation**.

Task isolation is the property that prevents a task from accessing memory regions and resources that do not belong to it. More specifically, a task should only be able to access:

- its own stack and local variables;
- explicitly shared memory regions;
- operating system services through controlled interfaces.

Without proper isolation, any task executing within the system may potentially inspect or modify data belonging to other tasks. Since the operating system cannot distinguish between legitimate and malicious accesses when all tasks share the same memory space, unauthorized operations may be performed without triggering any exception or warning.

The consequences of isolation failures can be quite severe: a malicious or compromised task may disclose confidential information stored by another task, alter application state, corrupt critical data structures, or interfere with the normal execution of the system. Depending on the nature of the targeted data, such violations may lead to denial-of-service conditions, integrity breaches, or privilege escalation.

For these reasons, task isolation represents one of the key security mechanisms in modern embedded systems. The ability to enforce clear boundaries between tasks is essential for protecting sensitive information and for preserving the correctness, reliability, and safety of the overall application.

1.2 Demonstration Purposes and Testing Repository

1.2.1 General Testing Environment Information

The implementation and testing of this project are based on **FreeRTOS v11.0.0**. The chosen hardware target is the **ARM Cortex-M3** processor, which provides the necessary hardware support for memory protection through its integrated MPU.

To facilitate development and demonstrate the attacks in a controlled environment, we utilize the **QEMU** emulator. The system is configured to emulate the **ARM MPS2 (AN385)** platform, allowing for precise observation of memory violations and task behavior as they would occur on physical hardware.

1.2.2 Folder Tree Structure

The repository is organized into a modular structure to separate the RTOS kernel, the hardware configuration, and the two different demonstration scenarios (with and without MPU).

```

group9-RTOS/
├── Demo/ .....Standard implementation without MPU protection
│   ├── main.c .....Vulnerable task implementations and attack logic
│   ├── FreeRTOSConfig.h .....Kernel configuration for non-MPU build
│   ├── startup.c .....System startup code
│   ├── uart.c .....UART driver implementation
│   ├── mps2_m3.ld .....Linker script for memory layout
│   └── Makefile .....Build system and QEMU execution scripts
├── Demo_MPU/ .....Hardened implementation utilizing the Hardware MPU
│   ├── main.c .....Secure task implementations using restricted APIs
│   ├── FreeRTOSConfig.h .....Kernel and MPU configuration for MPU build
│   ├── startup.c .....System startup code with MPU exception handler
│   ├── uart.c .....UART driver implementation
│   ├── mps2_m3.ld .....Linker script with .unprivileged_data section
│   └── Makefile .....Build system and QEMU execution scripts
├── Documentation/
│   └── Documentation.pdf .....The compiled project report
├── FreeRTOS-Kernel/ .....FreeRTOS v11.0.0 source code (Collapsed)
├── .gitignore .....Specifies intentionally untracked files to ignore
└── README.md .....Project overview and setup instructions

```

Note: All LaTeX source files and auxiliary configuration files have been omitted from the tree above to focus on the software implementation and the final documentation, as specified in the readme.

The repository of the project can be found here: <https://baltig.polito.it/cs4es2026/group9-RTOS>.

1.2.3 Demonstration Objectives

The primary objective of this project is to provide a comparative analysis between a standard RTOS execution environment and one hardened by hardware memory protection. Through a series of simulated scenarios, the ways in which the absence of isolation allows tasks to perform unauthorized memory operations was shown, giving as prime examples scanning for sensitive data within other tasks' memory or tampering with global state variables. The demonstration then illustrates how the integration of an MPU effectively mitigates these risks by enforcing strict access boundaries and confining task execution to authorized memory regions.

1.3 Ideal Scenario, Assumptions and Broken Rules

In a perfectly secure embedded environment, the system adheres strictly to the principle of least privilege. Every application task operates in unprivileged mode, confined within rigid memory boundaries enforced by the Hardware MPU. Communication between tasks is restricted and no task possesses the capability to directly inspect or modify the private stack or data regions of others. This architecture creates an environment where the compromise of a single task cannot lead to system-wide breaches, allowing for an overall quite robust structure.

1.3.1 Assumptions

To define the scope of our security analysis and the behavior of the emulated system, the system operates under the following assumptions:

- **Trusted Kernel:** the FreeRTOS kernel and its associated MPU wrappers are assumed to be bug-free and perfectly functioning under normal operating conditions.
- **Hardware Correctness:** the ARM Cortex-M3 processor and its MPU implementation are assumed to follow their specific architecture's specifications perfectly, without underlying side-channel or hardware-level vulnerabilities.
- **Secure Initialization:** the boot process is trusted. The system starts in a privileged state to configure the initial MPU regions and the RTOS scheduler correctly before dropping to unprivileged mode for task execution.
- **Inside Attacker Threat Model:** it is assumed that the attacker has already achieved code execution within a task (for example via a buffer overflow or logic error) in order to mostly focus on the ability of that task to escalate privileges or extract data from the rest of the system.

1.3.2 Broken Rules

To effectively demonstrate the vulnerabilities present in traditional RTOS deployments without hardware protection, the *Demo* implementation (Non-MPU) intentionally violates several critical security rules:

- **Flat Memory Model:** all tasks share a single global address space. There is no hardware enforcement of boundaries between the kernel and the various tasks.
- **Homogeneous Privilege:** every task executes with full processor privileges (Privileged Mode). This allows any task to disable interrupts, modify the vector table, or even reconfigure hardware peripherals directly.
- **Global Data Visibility:** sensitive variables (like cryptographic keys or shared counters) are placed in memory regions that are globally accessible for both reading and writing by any executing code.
- **Stack Introspection:** tasks are permitted to query the kernel for the memory locations of other tasks' stacks, facilitating the extraction of "private" data.

CHAPTER 2

Attack without the MPU

2.1 Step-by-Step Execution Scenario

Analyzing the `main.c` file, it's possible to identify three tasks:

- `priorityAndStackAttacker`
- `target`
- `globalVarAttacker`

2.1.1 Target task (victim)

This task has some local variables, among which `char pcSecret[ 65 ]` is the sensitive data: it's a key that the task uses to encrypt the string `char pcPassword`. To improve visualization, every 500 ms `target` checks if the key is the original one and logs on `stdout` a message according to the state of the secret. If the secret has been compromised, the task deletes itself and a new instance (with the correct secret) is created. To keep track of the current check number of `target`, a global variable `ulSharedCounter` is used.

2.1.2 Global Data Attacker task

This task aims to alter the content of the global variable `ulSharedCounter`, setting it to 9999. This is meant to demonstrate how a supposed unauthorized task can change global variables' values.

2.1.3 Stack Attacker task

The task `priorityAndStackAttacker` initially waits for two seconds, and then starts its first attack increasing its own priority (from 1 to 7). At this point, with a period of two seconds, it executes some privileged FreeRTOS functions to access the stack of other tasks:

`uxTaskGetNumberOfTasks()`

```
1 UBaseType_t uxNumberOfTasks = uxTaskGetNumberOfTasks();
```

Returns the number of tasks that the RTOS kernel is currently managing.

`vTaskGetInfo()`

```
1 vTaskGetInfo( NULL, &xCurTaskInfo, pdFALSE, eInvalid );
```

Puts on `TaskStatus_t xCurTaskInfo` some information about `priorityAndStackAttacker` (with `highWaterMark` excluded because of `pdFALSE` parameter)

uxTaskGetSystemState()

```
1      UBaseType_t uxNumberOfTasks = uxTaskGetSystemState(pxTasksManaged, uxNumberOfTasks,
      NULL);
```

Puts on `TaskStatus_t pxTasksManaged[ uxNumberOfTasks ]` some information about all tasks managed by the kernel.

To better understand how powerful these instructions are, it's useful to observe how the `struct` variable `TaskStatus_t` is organized (`task.h:167`).

```
1      typedef struct xTASK_STATUS
2      {
3          TaskHandle_t xHandle;
4          const char * pcTaskName;
5          UBaseType_t xTaskNumber;
6          eTaskState eCurrentState;
7          UBaseType_t uxCurrentPriority;
8          UBaseType_t uxBasePriority;
9          configRUN_TIME_COUNTER_TYPE ulRunTimeCounter;
10         StackType_t * pxStackBase;
11         #if ( ( portSTACK_GROWTH > 0 ) || ( configRECORD_STACK_HIGH_ADDRESS == 1 ) )
12             StackType_t * pxTopOfStack;
13             StackType_t * pxEndOfStack;
14         #endif
15         configSTACK_DEPTH_TYPE usStackHighWaterMark;
16         #if ( ( configUSE_CORE_AFFINITY == 1 ) && ( configNUMBER_OF_CORES > 1 ) )
17             UBaseType_t uxCoreAffinityMask;
18         #endif
19     } TaskStatus_t;
```

By changing some settings of `FreeRTOSConfig.h`, in particular adding the line

```
1      #define configRECORD_STACK_HIGH_ADDRESS 1
```

`TaskHandle_t` can also store the pointers to the Top and the End of Stack.

Task Stack Occupation

To better explain, let's roughly consider a task's stack anatomy in FreeRTOS. It's allocated in the heap region of the RAM, and grows towards low addresses. To keep track of the stack usage FreeRTOS provides three pointers in the `TaskStatus_t` structure:

- `StackType_t *pxEndOfStack`: pointer to the highest address of the stack
- `StackType_t *pxStackBase`: pointer to the lowest address of the stack
- `StackType_t *pxTopOfStack`: pointer to the current top of stack (saved SP)

Stack is initially filled with hex pattern `0xA5`, so it's possible to identify unused portions. Hence, by iterating among all elements in the `pxTasksManaged` array, `priorityAndStackAttacker` can obtain the stack boundaries of `target` and analyze the content of the stack to find the secret key.

To have an idea about `priorityAndStackAttacker`'s point of view regarding `target`'s stack, let's perform some debugging operations with gdb. First, let's run the demo to visualize the stack boundaries of `target`:

```
1      +++ Analyzing Stack for Task: target +++
2      [priorityAndStackAttacker] Stack Base: 0x20000ac8
3      [priorityAndStackAttacker] Stack Top:  0x20001158
4      [priorityAndStackAttacker] Stack End:  0x200012c4
```

Then we can dump the content of the stack in a binary file

```
1      (gdb) dump binary memory before.bin 0x20000AC8 0x200012C4
```

And finally analyze it with hexdump.

```

1  > hexdump -C before.bin
2
3  00000000  a5 a5 a5 a5 a5 a5 a5 a5  a5 a5 a5 a5 a5 a5 a5 |.....|
4  *
5  00000550  a5 a5 a5 a5 a5 a5 a5 a5  a8 10 00 20 10 00 00 00 |.....|
6  00000560  04 00 00 20 38 11 00 20  cd 04 00 00 cd 04 00 00 |... 8..|
7  00000570  74 10 00 20 e3 0c 00 00  38 11 00 20 a8 10 00 20 |t... 8..|
8  00000580  a8 10 00 20 10 00 00 00  04 00 00 20 38 11 00 20 |... 8..|
9  00000590  cd 04 00 00 e9 10 00 20  05 00 00 00 03 0e 00 00 |.....|
10 000005a0  cd 04 00 00 f5 04 00 00  fa 0a 00 00 02 00 00 00 |.....|
11 000005b0  01 00 00 00 8d 12 00 00  c8 13 00 00 01 00 00 00 |.....|
12 000005c0  02 00 00 00 38 11 00 20  04 00 00 20 d9 07 00 00 |...8..|
13 000005d0  a4 10 00 20 a5 a5 a5 a5  a5 a5 a5 a5 b0 11 00 20 |...|
14 000005e0  04 00 00 00 ff ff ff ff  ff ff ff ff 02 00 00 00 |.....|
15 000005f0  02 00 00 00 03 00 00 00  78 20 30 a5 a5 a5 a5 |.....x 0....|
16 00000600  a5 a5 a5 a5 a5 a5 a5 a5  a5 a5 a5 a5 a5 a5 a5 |.....|
17 *
18 00000620  a5 35 32 00 a5 78 a5 a5  a5 a5 a5 a5 a5 a5 a5 |.52..x.....|
19 00000630  a5 a5 a5 a5 a5 a5 a5 a5  00 00 00 00 a5 a5 a5 a5 |.....|
20 00000640  a5 a5 a5 a5 04 00 00 20  4c 13 00 00 a5 a5 a5 a5 |.....L.....|
21 00000650  b8 11 00 20 a5 a5 a5 a5  a5 a5 a5 a5 a5 a5 a5 |...|
22 00000660  a5 a5 a5 a5 7f 04 00 00  a5 a5 a5 a5 ac 11 00 20 |.....|
23 00000670  2f 12 00 20 a5 a5 a5 a5  7c 00 00 00 08 02 ff ff |/.|.....|
24 00000680  2c 12 00 20 7f 00 00 00  a5 a5 a5 a5 a5 a5 a5 |,...|
25 00000690  e4 11 00 20 4c 13 00 00  a5 a5 a5 a5 98 11 00 20 |...L.....|
26 000006a0  a5 a5 a5 a5 a5 a5 a5 a5  a5 a5 a5 a5 a5 a5 a5 |.....|
27 000006b0  00 00 00 00 d4 12 00 20  00 00 00 10 04 ed 00 e0 |.....|
28 000006c0  ff a3 a4 a4 af 26 00 00  e2 20 00 00 00 00 00 61 |....&...a|
29 000006d0  44 49 00 00 f4 01 00 00  e4 11 00 20 00 00 00 00 |DI.....|
30 000006e0  b8 11 00 20 95 48 00 00  a5 a5 a5 a5 a5 a5 a5 |...H.....|
31 000006f0  a5 a5 a5 a5 00 00 00 00  31 04 59 4b 58 42 0b 10 |.....1.YKXB..|
32 00000700  5d 22 51 45 02 57 57 52  00 a5 a5 a5 53 65 6e 73 |]"QE.WWR....Sens|
33 00000710  69 74 69 76 65 44 61 74  61 31 32 33 00 a5 a5 a5 |itiveData123....|
34 00000720  62 61 37 38 31 36 62 66  38 66 30 31 63 66 65 61 |ba7816bf8f01cfea|
35 00000730  34 31 34 31 34 30 64 65  35 64 61 65 32 32 32 33 |414140de5dae2223|
36 00000740  62 30 30 33 36 31 61 33  39 36 31 37 37 61 39 63 |b00361a396177a9c|
37 00000750  62 34 31 30 66 66 36 31  66 32 30 30 31 35 61 64 |b410ff61f20015ad|
38 00000760  00 a5 a5 a5 35 32 20 00  6b 42 5d 20 43 79 63 6c |....52 .kB] Cyc1|
39 00000770  65 20 34 0a 00 a5 a5 a5  a5 a5 a5 a5 a5 a5 a5 |e 4.....|
40 00000780  a5 a5 a5 a5 a5 a5 a5 a5  a5 a5 a5 a5 a5 a5 a5 |.....|
41 *
42 000007d0  a5 a5 a5 a5 a5 a5 a5 a5  00 00 00 00 a5 a5 a5 a5 |.....|
43 000007e0  a5 a5 a5 a5 10 00 10 00  a5 a5 a5 a5 a5 a5 a5 |.....|
44 000007f0  a5 a5 a5 a5 5d 40 00 00  a5 a5 a5 a5 |....]@.....|
45 000007fc

```

At this point, the code is easy to understand, because it's a while loop starting from `pxEndOfStack` (`pucEndOfStack`), until `pxTopOfStack` (`pucTopOfStack`) is reached. The stack region from `pxTopOfStack` to `pxStackBase` is unused (`0xA5`). The loops searches for the longest sequence of printable bytes:

```

1  if( ucByteFromMem >= 32 && ucByteFromMem <= 122 ).

```

And when it's found, to aim exactly at `target`'s key, the sequence whose length is equal to the size of `pcSecret` (in this case corresponding to 64) is corrupted by randomly altering its content.

2.2 Result of Task Isolation Failure

As expected (or not), all operations are considered to be legitimate by the system, so both attackers succeed in their final objective. To be more specific, the memory dump of **target**'s stack after the attack is shown, together with a screenshot of the terminal executing QEMU.

```

1  > hexdump -C after.bin
2
3      00000700  5d 22 51 45 02 57 57 52  00 a5 a5 a5 53 65 6e 73  |]"QE.WWR....Sens|
4      00000710  69 74 69 76 65 44 61 74  61 31 32 33 00 a5 a5 a5  |itiveData123....|
5      00000720  32 51 70 31 50 6f 30 4f  6e 2f 4e 6d 2e 4d 6c 2d  |2Qp1Po00n/Nm.Ml-|
6      00000730  4c 6b 2c 4b 6a 2b 4a 69  2a 49 68 29 48 67 28 47  |Lk,Kj+Ji*Ih)Hg(G|
7      00000740  66 27 46 65 26 45 64 25  44 63 24 43 62 23 42 61  |f'Fe&Ed%Dc$Cb#Ba|
8      00000750  22 41 60 21 40 5f 7e 3f  5e 7d 3e 5d 7c 3d 5c 7b  |"A'!@_~?}>]|=\{|
9      00000760  00 a5 a5 a5 35 32 20 00  6b 42 5d 20 43 79 63 6c  |....52 .kB] Cyc1|
10     00000770  65 20 34 0a 00 a5 a5 a5  a5 a5 a5 a5 a5 a5 a5  |e 4.....|

```

```

1  > make qemu_start
2
3      [target] Cycle 1
4      [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
5      [target] Key integrity OK
6      [target] Cycle 2
7      [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
8      [target] Key integrity OK
9      [target] Cycle 3
10     [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
11     [target] Key integrity OK
12     [target] Cycle 4
13     [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
14     [target] Key integrity OK
15
16     [priorityAndStackAttacker] priorityAndStackAttacker priority increased!
17
18     +++ Analyzing Stack for Task: target +++
19     [priorityAndStackAttacker] Stack Base: 0x20000ac8
20     [priorityAndStackAttacker] Stack Top: 0x20001158
21     [priorityAndStackAttacker] Stack End: 0x200012c4
22     [priorityAndStackAttacker] LONGEST STRING:
23     [priorityAndStackAttacker] Addr: 0x200011e8 (Len: 64)
24     [priorityAndStackAttacker] Data:
25         ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
26     [priorityAndStackAttacker] Memory Corruption : Corrupting target's string in its
27         stack!
28
29     [globalVarAttacker] GLOBAL DATA ATTACK: ulSharedCounter forced to 9999
30
31     [target] Cycle 10000
32     [target] Encrypted: 61 34 1e 42 39 1b 59 39 0b 6b 2f 19 4f 7c 5e 1e
33     [target] WARNING: Key has been corrupted! Respawning a clean target...
34     [target] Cycle 1
35     [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
36     [target] Key integrity OK

```

CHAPTER 3

Implementation of the MPU

3.1 What is an MPU?

An **MPU** (Memory Protection Unit) is a HW component dedicated to managing access to memory regions, following the directions imparted by a kernel-level process. This process assigns to each process one or more memory regions with specific access permissions (read, write, execute).

The MPU is **interposed** between the CPU and the memory: when a process asks to access a specific portion of memory, the MPU takes the call and checks the privilege level of the CPU, to verify if the process is allowed to access it. Two outcomes are possible:

- **Access allowed:** the MPU forwards the call to the memory and returns the result to the process.
- **Access not allowed:** the MPU raises a **MemManage** exception and the handler is called.

Using an MPU increases the **security and safety** of the system, as it prevents processes from accessing memory regions they are not allowed to, which can lead to data corruption, data leakage or even code execution by malicious third parties. On the other hand, it also introduces some **overhead**, as the MPU needs to check every memory access and raise exceptions when necessary, which can lead to performance degradation if not used properly or system crashes if the task is not well designed and the handler does not manage the system restore. This introduces a critical challenge in **real-time systems**, where the correct working of a system depends on strict execution timing.

FreeRTOS provides a software wrapper of the MPU for the ARM Cortex-M3 architecture in the file **MPU_Wrapper.h**, which allows to **manage memory regions and access permissions** for each task in the application code, abstracting the register-level MPU configuration from the user. The wrapper provides functions to create unprivileged tasks (User Mode), set memory regions for each task specifying permissions, and handle exceptions raised by the MPU when a task tries to access a memory region it is not allowed to.

Tasks have two different privilege levels: privileged, also known as **Privileged Mode**, and unprivileged, also known as **User Mode**. Privileged tasks usually have access to all memory regions (if not specified otherwise) and can execute any instruction, while user tasks have limited access to memory regions and can only execute a subset of instructions. By **default**, all tasks are created as privileged, but we can use the MPU wrapper functions to create unprivileged tasks and assign them specific memory regions with defined permissions. MPU regions can be configured to prevent User mode access, while still permitting Privileged mode access.

Since unprivileged tasks running in User Mode are strictly **barred** from directly accessing kernel data structures and hardware peripherals, they cannot invoke standard FreeRTOS API functions conventionally. To bypass this restriction safely, FreeRTOS-MPU utilizes a system call mechanism based on the ARM Cortex-M **SVC** (Supervisor Call) instruction.

When an unprivileged task calls an MPU-wrapped API function (i.e., **vTaskDelay**), the wrapper generates an **SVC** exception, passing a specific system call number. This triggers a hardware-level transition:

1. The CPU suspends the unprivileged task and enters **Handler Mode**, which automatically grants **Privileged** permissions.
2. The **vPortSVCHandler** intercepts the exception, identifies the requested API via the system call number, and validates the memory pointers passed as arguments to prevent privilege escalation exploits.
3. The kernel executes the operation on behalf of the unprivileged task, switches the CPU back to **User Mode (Unprivileged)**, and returns execution to the task.

Regions are characterized by a base address, a size and access permissions, with support for up to 8 regions per task. By modifying the `startup.c` file we can also declare a **handler** for the MPU exception, so that we can specify what will happen when a task violates memory boundaries.

This strict isolation requires the system configuration to be dynamic, which directly impacts how the **FreeRTOS Scheduler** handles context switches. The MPU hardware possesses a limited number of region registers; therefore, memory boundaries must be **reconfigured** on the fly every time a new task gains control of the CPU.

During a **context switch**, the scheduler performs the following steps:

- It saves the CPU context (registers) of the currently running task onto its private stack.
- It accesses the Task Control Block (TCB) of the next task selected for execution to retrieve its specific memory topology.
- It reprograms the MPU hardware registers (MPU->RBAR and MPU->RASR), wiping out the memory permissions of the old task and writing the base addresses, sizes, and attributes of the new task's stack and custom regions.
- It restores the CPU context of the new task.

As soon as the processor exits the scheduler handler, the new MPU configuration becomes active. This ensures that even during rapid multitasking, no task can ever breach the memory domain of another.

3.2 Unit Implementation

3.2.1 First implementation and issues with global variables

To emulate a use case in a MPU-protected environment, we started with the non-MPU implementation of the previous chapter and we modified it to support the MPU features.

- **FreeRTOSConfig.h**: adding the necessary definitions to enable the MPU
 - `configENABLE_MPU = 1`: this enables the MPU support in FreeRTOS.
 - `configUSING_MPU_WRAPPERS_V1 = 1` and `configUSE_MPU_WRAPPERS_V1 = 1`: this enables the use of the MPU wrapper functions in the application code.
 - `configENABLE_ACCESS_CONTROL_LIST = 0`: this disables the use of access control lists, which is not needed for our implementation.
 - `configPROTECTED_KERNEL_OBJECT_POOL_SIZE = 150`: this sets the size of the pool of protected kernel objects, which is needed for the MPU support.
 - `configSYSTEM_CALL_STACK_SIZE = 128`: this sets the size of the system call stack.
 - `configENFORCE_SYSTEM_CALLS_FROM_KERNEL_ONLY = 1`: this enforces that system calls can only be made from kernel mode.
 - `configALLOW_UNPRIVILEGED_CRITICAL_SECTIONS = 0`: this disallows unprivileged critical sections.
- **Makefile**: adding the includes for the MPU wrapper.
- **main.c**: declaring memory regions and modifying task creation by using `xTaskCreateRestricted` instead of `xTaskCreate`, to specify the memory regions and permissions for each task.

The first problem encountered revolved around tasks being unable to access global variables, which were declared in the `.data` section and were not included in the memory regions assigned to the tasks.

This problem was debugged by defining the `MemManage` handler and hook, so that it would print the following informations:

- The error type, which is kept in the `CFSR->MMFSR` register, to understand what kind of memory access violation occurred. The specific error (such as a Data Access Violation, `DACCVIOL`) can be inferred by checking the bits of the register through a bitwise AND operation and cross-referencing the table in the ARM Cortex-M3 documentation.
- The address that was trying to be accessed, which is kept in the `MMFAR` register. This register can be safely read only if the `MMARVALID` bit of the `CFSR->MMFSR` register is explicitly set by the hardware.
- The address of the instruction that caused the violation, which is kept in the `PC` register (extracted from the exception stack frame).

We then used the information printed by the handler to understand where the violation was happening using GDB, finding that the culprit was the access to the global variable `ulSharedCounter` in the tasks, which was not included in the memory regions assigned to the tasks.

To isolate and protect this variable under the MPU while complying with hardware rules, the variable's allocation must satisfy two strict mathematical constraints enforced by the Cortex-M MPU architecture:

1. The size of any mapped region must be a strict power of two (2^N), with a minimum size of 32 bytes.
2. The base address of the region must be an exact multiple of its size (natural alignment).

Therefore, in our implementation, `ulSharedCounter` is aligned to a 32-byte boundary (the minimum MPU region size supported) using the compiler attribute:

```
1 volatile uint32_t ulSharedCounter __attribute__(( aligned( 32 ) )) = 0;
```

During task creation, the **target** task is explicitly granted read/write access to this 32-byte memory region via its MPU region parameters configuration:

```
1 TaskParameters_t targetParams =
2 {
3     ...
4     .xRegions =
5     {
6         { ( void* )&ulSharedCounter, 32, portMPU_REGION_READ_WRITE },
7         { 0, 0, 0 },
8         { 0, 0, 0 }
9     }
10 };
```

It is important to note that in the current implementation shown in **main.c** the aforementioned global variable **ulSharedCounter** is not removed completely. It is intentionally kept as a shared object, but it is mapped only in the MPU region assigned to **target**. This choice is useful for the demonstration because it shows that the MPU does not protect data merely by its global status, but by the precise permissions granted to each task. In other words, **target** can access the shared value, while **globalVarAttacker** is denied access if its region list does not include that memory area.

3.2.2 Second implementation and issues with **snprintf**

After solving the problem with the global variables, another problem was encountered, this time having to do with the use of the **snprintf** function, which was extensively used in the tasks.

The issue was again found through debugging using the information given by the **MemManage** handler, which showed that the violation was happening in the **snprintf** function, as it was trying to access memory regions that were not included in the task's memory regions.

In fact, the **snprintf** function is not a simple function that only accesses the stack, since it also makes use of global status variables, along with the heap for dynamic memory allocation, which is not included in the task's memory regions.

This meant that it was necessary to find a way to use a function similar to **snprintf** that would not access memory regions outside of the task's stack, and different methods were tried to achieve such result. The reason this matters is that the standard C library is not fully MPU-friendly: its implementation may rely on the internal global state, the heap, or other memory areas that are not part of the task-local region map.

- An **snprintf_mpu** function was written, which only used local variables and did not access any global variable or dynamic memory allocation. This function was able to print strings as well as the standard **snprintf**, but was way too complex in its implementation and not very efficient, as it had to handle all the formatting and string manipulation manually, without the use of any library function.
- Another custom implementation using three functions was tried, all grouped in the **print_helper** file:
 - **print_int**: this function takes an integer as input and prints through UART the string equivalent of the given argument.
 - **print_ptr**: this function takes a pointer as input and prints through UART the string equivalent of the given argument.
 - **print_hex**: this function takes an hex value as input and prints through UART the hexadecimal string equivalent of the given argument.

- Finally, it was discovered that the standard library **Newlib**, which is a C library implementation specific for ARM embedded systems, provides a function called `_snprintf_r`, a reentrant version of `snprintf` that does not use any global variable and does not perform any dynamic memory allocation.

Instead of using global variables, it uses a reentrancy structure that is passed as an argument to the function, which contains all the necessary information for the function to work, such as the buffer to write to and the size of the buffer. The struct is then allocated on the stack of the task, so it is included in the task's memory regions and does not cause any violation when accessed by the function. Implementing such a structure appeared to be easier and more straightforward than the alternative solutions proposed, since it only required the following steps:

1. Modify the Makefile so that the `_snprintf_r` function is included in the compilation.
2. Include the `_snprintf_r` function in the `main.c`.
3. Replace all the calls to `snprintf` with calls to `_snprintf_r`.
4. Instance the reentrancy structure, initializing it through the `_REENT_INIT_PTR` function and passing it to `_snprintf_r` as an argument.

The actual initialization and call pattern in our task implementation is structured as follows:

```
1 struct _reent xLocalReent;  
2 _REENT_INIT_PTR(&xLocalReent);  
3 _snprintf_r(&xLocalReent, pcWriteBuffer, xWriteBufferLen, ...);
```

3.2.3 Final working implementation

After solving the problem with the `snprintf` function, it was possible to run the virtual machine with the MPU support enabled, seeing the tasks running and printing their messages through UART, with a demonstration of the memory protection provided by the MPU, as the tasks were not able to access memory regions outside theirs.

A further point that is worth highlighting from the code comments is that the demo is intentionally built as an attacker-style scenario. `priorityAndStackAttacker` uses FreeRTOS introspection APIs to inspect the stack of the other tasks and to search for printable strings, while `target` and `globalVarAttacker` represent two different execution paths: one legitimate task with a mapped shared region and one task that attempts to write to memory that is not granted to it. This part of the implementation is not an MPU primitive by itself; it is a demonstration of how the MPU, combined with careful region design, can contain unwanted accesses and trigger a `MemManage` fault when a region boundary is violated.

3.3 Overview of Stack after Implementation

Following the successful integration of the hardware MPU features and the migration to reentrant library functions, the layout of the system's RAM undergoes a significant architectural formalization. This section details how the stack topology is structured to guarantee execution safety and analyzes the behavioral outcomes of the implemented attack vectors.

3.3.1 Linker Script Isolation and the Power-of-Two Alignment Rule

To allow unprivileged tasks to execute within a restricted memory window, the linker script `mps2_m3.ld` was modified to isolate task stacks into a dedicated memory section called `.unprivileged_data`. In `(main.c)`, the stacks are instantiated statically as arrays of `StackType_t` and bound to this section using the compiler directive:

```
1  __attribute__(( section(".unprivileged_data"), aligned(STACK_SIZE_BYTES) ))
```

The inclusion of the `aligned` attribute is a critical hardware dependency dictated by the ARM Cortex-M3 MPU design. As introduced in Section 3.2.1, the hardware MPU enforces strict constraints regarding region size (a power of two) and base address (natural alignment). Since each task's stack has a configured size of 2048 bytes (which is a power of two), setting proper natural alignment is mandatory for the MPU to calculate the protected memory window correctly, as also documented in the [FreeRTOS MPU region documentation](#). If this explicit alignment was omitted, the linker might allocate the array at an arbitrary boundary, causing the MPU hardware to miscalculate the protected window, which would trigger immediate `MemManage` faults upon context switching.

3.3.2 MPU Enforcement and Attack Vector Demonstration

The runtime behaviour of the system, as observed through UART outputs and the MPU exception handler, confirms that the MPU is effectively enforcing memory access permissions.

Even if the memory section is shared, the MPU manages to isolate the tasks by only allowing access to the specific regions assigned to each task, eliminating the risks of stack overflows and memory corruption between tasks, as each task can only access its own stack and the shared region if it is assigned to it.

In the case of `priorityAndStackAttacker`, this point was demonstrated exactly by trying to access the stack of `target` to search and corrupt the cryptographic key, which is a critical DoS attack vector in real-world scenarios.

In the case of `globalVarAttacker`, the demonstration instead revolved around how the MPU can prevent unauthorized access to global variables, which is a common attack vector for data corruption and privilege escalation.

3.3.3 `_snprintf_r` and the Stack Footprint

Finally, the implementation of Newlib's `_snprintf_r` impacts the internal stack footprint on a structural level. Because the reentrancy structure (`struct _reent local_reent`) is declared as a local variable within the task's execution loop, it is allocated directly onto the active stack frame.

By locking all operational variables within the boundary of this single, power-of-two aligned stack array, the firmware guarantees that intensive string formatting operations proceed safely within the localized sandbox authorized by the MPU hardware.

CHAPTER 4

Attack with the MPU

4.1 Attack Implementation

4.1.1 First Attack: Stack Scanning and TCB Introspection Failure

In the MPU-enabled configuration, all three defined tasks (`priorityAndStackAttacker`, `target`, and `globalVarAttacker`) run in unprivileged Thread mode, and as a consequence any access to privileged memory will trigger a `MemManage` fault. To execute the stack-scanning attack, `priorityAndStackAttacker` calls the API `uxTaskGetSystemState` to retrieve the list of active tasks. It then iterates over the returned task descriptors to identify target tasks using the following comparison:

```
1 strcmp(pxTasksManaged[uxIndex].pcTaskName, xCurrentTaskInfo.pcTaskName) != 0
```

In FreeRTOS, task names and other metadata are stored inside the Task Control Blocks (TCBs), which are allocated by the kernel and reside within privileged RAM. Because `priorityAndStackAttacker` is unprivileged and its MPU regions (`xRegions`) do not map the privileged memory containing the TCBs, any attempt to dereference the pointer `pxTasksManaged[ uxIndex ].pcTaskName` within `strcmp` violates MPU permissions. The CPU immediately detects this unauthorized read access and triggers a `MemManage` fault, stopping `priorityAndStackAttacker` before it can inspect or corrupt `target`'s stack.

4.1.2 Second Attack: Shared Global Variable Access Protection

`globalVarAttacker` attempts to perform a global data modification attack by writing directly to the shared global variable:

```
1 ulSharedCounter = 9999;
```

To isolate and protect this variable, it is aligned to a 32-byte boundary and assigned to `target`'s MPU region list as detailed in Section 3.2.1. On the other hand, `globalVarAttacker` is created with an empty region mapping configuration (`{ 0, 0, 0 }`). Therefore, when `globalVarAttacker` attempts to write to `ulSharedCounter`, the MPU blocks the operation, instantly triggering a `MemManage` fault.

4.1.3 Transition from Dynamic to Static Task Buffer Allocation and MPU Setup

Comparing the MPU demo with the standard non-MPU demo, several structural code modifications were required to comply with hardware isolation rules:

- **Static Memory Allocation:** in the non-MPU demo, `priorityAndStackAttacker` allocated memory dynamically on the heap:

```
1 pxTasksManaged = pvPortMalloc( uxNumberOfTasks * sizeof( TaskStatus_t ) );
```

Under MPU rules, the heap is privileged. Calling `pvPortMalloc()` from an unprivileged task like `priorityAndStackAttacker` attempts to modify heap metadata in privileged RAM, violating MPU

permissions and triggering a `MemManage` fault. Safe task execution requires defining stack buffers statically. As explained in the [FreeRTOS API documentation for restricted task creation](#), creating unprivileged tasks with predefined regions prevents unauthorized heap accesses.

To solve this, the buffer was declared statically as a local array on the task's stack:

```
1 TaskStatus_t pxTasksManaged[ 10 ];
```

Since the task's stack is automatically mapped as unprivileged-writable by the MPU, this prevents authorization faults.

- **Stack Alignment and Placement:** task stacks in the MPU demo must be aligned to their total size (a power of 2) and placed in the dedicated `.unprivileged_data` section as detailed in Section 3.3.1, instead of being declared as standard unaligned arrays.
- **Reentrant Formatting:** unprivileged tasks use the reentrant version of the standard formatting function, `_snprintf_r` (detailed in Section 3.2.2), to perform string formatting on their own stack frames, avoiding unauthorized access to privileged global status variables.
- **Restricted Task Creation:** tasks in the MPU demo are initialized via the API `xTaskCreateRestricted` instead of the standard `xTaskCreate` function. This API requires passing pre-allocated, aligned stacks and defining specific MPU region permissions (`xRegions`) for each task.

4.2 Step-by-Step Execution Scenario

Under QEMU, the execution scenario follows these distinct phases:

1. **System Initialization:** The UART interface is initialized, and the MPU descriptors for all three tasks are loaded. When the scheduler starts, the CPU loads the appropriate MPU region configuration on every context switch.
2. **Target Task Normal Operation:** `target` starts executing. It regularly increments the shared counter, performs XOR encryption, and verifies the integrity of the secret key.
3. **priorityAndStackAttacker Wakeup:** After 2 seconds, `priorityAndStackAttacker` wakes up, increases its priority to prevent other tasks from running, and calls `uxTaskGetSystemState` to locate other active tasks.
4. **priorityAndStackAttacker MPU Violation:** As soon as `priorityAndStackAttacker` attempts to read the task name of `target` inside the `strcmp` routine (address `0x200002C0`), the MPU blocks the read attempt, triggering a `MemManage` fault.
5. **globalVarAttacker MPU Violation (Simulated with priorityAndStackAttacker disabled):** If `priorityAndStackAttacker` is disabled, `target` continues running. After 3 seconds, `globalVarAttacker` wakes up, prints a message, and attempts to write to `ulSharedCounter`. The MPU blocks this write attempt at address `0x20008080`, triggering a `MemManage` fault.

4.3 Result of Task Isolation Failure

When the MPU is active, task isolation is enforced at the hardware level. The unauthorized memory accesses are intercepted before they can corrupt other tasks' memory or global variables. Instead, the CPU freezes execution within the `MemManage_Handler` to prevent privilege escalation.

4.3.1 priorityAndStackAttacker Exception Log

When `priorityAndStackAttacker` tries to read `target`'s name from its TCB, the MPU halts the CPU and logs the following report in the QEMU console:

```

1 [SYSTEM] UART initialized.
2 [SYSTEM] Starting Scheduler...
3 [target] Cycle 1
4 [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
5 [target] Key integrity OK
6 [target] Cycle 2
7 [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
8 [target] Key integrity OK
9 [target] Cycle 3
10 [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
11 [target] Key integrity OK
12 [target] Cycle 4
13 [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
14 [target] Key integrity OK
15
16 [priorityAndStackAttacker] priorityAndStackAttacker priority increased!
17
18 =====
19 !!! MPU VIOLATION DETECTED: MemManage Fault !!!
20 =====
21 Instruction pointer (PC) that triggered fault: 0x00011204
22 Violated Memory Address (MMFAR):                0x200002C0
23 Violation Type: DACCVIOL (Data Read/Write access violation)
24 =====
25 [SECURITY] Freezing system to prevent privilege escalation.

```

The fault is triggered at instruction pointer PC 0x00011204 (inside `strcmp`) due to reading the privileged address 0x200002C0 (the name string of `target` inside its TCB).

4.3.2 globalVarAttacker Exception Log

When `globalVarAttacker` attempts to write to `ulSharedCounter` (with `priorityAndStackAttacker` disabled), the MPU generates the following log:

```

1 [SYSTEM] UART initialized.
2 [SYSTEM] Starting Scheduler...
3 [target] Cycle 1
4 [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
5 [target] Key integrity OK
6 ...
7 [target] Cycle 6
8 [target] Encrypted: 31 04 59 4b 58 42 0b 10 5d 22 51 45 02 57 57 52
9 [target] Key integrity OK
10
11 [globalVarAttacker] GLOBAL DATA ATTACK: Attempting to modify ulSharedCounter...
12
13 =====
14 !!! MPU VIOLATION DETECTED: MemManage Fault !!!
15 =====
16 Instruction pointer (PC) that triggered fault: 0x00011B3E
17 Violated Memory Address (MMFAR):                0x20008080
18 Violation Type: DACCVIOL (Data Read/Write access violation)
19 =====
20 [SECURITY] Freezing system to prevent privilege escalation.

```

Here, the fault is caught at PC 0x00011B3E (an instruction inside `globalVarAttacker`) when writing to address 0x20008080 (the aligned memory location of `ulSharedCounter`).

CHAPTER 5

Conclusions

5.1 Summary of the Experience

The development of this project provided our group with a comprehensive understanding of the critical role task isolation plays in secure embedded systems. Throughout this experience, we encountered several significant technical challenges that deepened our expertise:

- **Low-Level Memory Management:** configuring the MPU required us to strictly adhere to hardware specific rules, such as the natural alignment of regions to power of two sizes. This gave us the chance to look further into linker scripts (`mps2_m3.ld`) and static allocation strategies using compiler attributes.
- **Kernel Introspection and Security:** implementing the attack vectors (stack scanning and global data corruption) highlighted how inherently vulnerable a standard RTOS environment can be. Successfully triggering `MemManage` faults in the `Demo_MPU` version proved the effectiveness of hardware-assisted sandboxing.
- **Standard Library Compatibility:** debugging the memory violations caused by the standard `snprintf` function among the most vital parts of the projects. The lesson it taught us is that making a safer and more robust system often requires moving away from common development patterns in favor of MPU-friendly, reentrant library functions.
- **Emulation and Debugging:** utilizing QEMU for testing allowed us to observe memory violations in real-time. Tracing the exception stack frame to identify the exact instruction causing a fault was very important to properly understand how low-level system debugging works.

In conclusion, this project demonstrated that hardware-assisted isolation introduces additional complexity during the configuration and build phases, but coming at the benefit of much better security, indispensable for modern embedded applications handling sensitive data or cryptographic material.

5.2 Sources and References

The following resources were consulted during the development and documentation of this project:

- General guide on FreeRTOS's memory management system: www.digikey.it/memory-management
- How FreeRTOS's MPU works: elektroda.freertos.org/media/MPU_Chapter.pdf
- Allocation of global variables when using the MPU: www.freertos.org/vTaskAllocateMPURegions
- Static vs Dynamic allocation when using the MPU: www.freertos.org/xTaskCreateRestricted
- Registers for the MPU: developer.arm.com/documentation/memory-protection-unit
- MemViolation message error code: developer.arm.com/configurable-fault-status-register